EPFL - Computer Science

CS-401 Applied Data Analysis

Lecture 02 — September 17, 2025

Author: Jonathan Dutoit
Professor: Maria Brbic

Handling Data

Lecture overview

1. Data Models
2. Data Sources
3. Data wrangling

Data model

"A data model is an abstract model that organizes elements of data and standardizes how they relate to one another and to the properties of real-world entities..." - Wikipedia

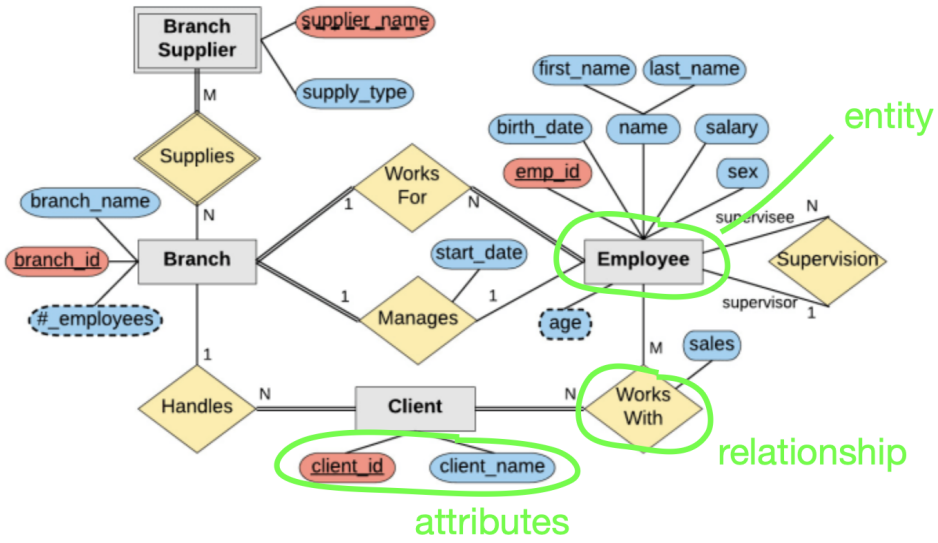
Simple definition

A data model specifies how you think about the world.

For instance, hospital reports. A patient comes to a hospital and it's record is saved. You can link all data to relationships between patients and doctors.

Q: "How do you think about the world?"

A: "See my entity-relationship diagram!"



How to store my data on a computer?

Q1: "How should I store my data on a computer?"

Q2: "How do I think about the world?"

- **Flat model**: The world is simple, one type of entity, all with the same attributes
- **Relational model**: The world contains many types of entities, connected by relationships
- **Document model**: The world is a hierarchy of entities
- **Network model**: The world is a complex network of entities

Flat Model

"The world is simple: one type of entity, all with the same attributes"

- Example: log files; e.g., Apache web server (httpd)
  - Entities = request from clients to server
- Another common format: CSV ("comma-separated vector")

Relational Model

"The world contains many types of entities, connected by relationships"

- The relational model is ubiquitous:
  - MySQL, PostgreSQL, Oracle, $DB_2, SQLite, ...$
  - You use it many times every day

- Data represented as tables describing
  - entities
  - relationship between entities

Processing data in the relational model: SQL (Structured Query Language)

- Declarative language for core data manipulations
- You think about what you want, not how to compute it

SQL

- You should know basics of SQL
- Key operations
  - Select (!), update, delete
  - Unique keys
  - Joins (inner, left outer, right outer, full) Sorting
  - Aggregation (group by, count, min, max, avg, etc.)

SQL Implementation

```
#!/usr/bin/python

import MySQLdb

# Open database connection
db = MySQLdb.connect("localhost","testuser","test123","TESTDB" )

# prepare a cursor object using cursor() method
cursor = db.cursor()

sql = "SELECT * FROM EMPLOYEE \
       WHERE INCOME > '%d'" % (1000)
try:
   # Execute the SQL command
   cursor.execute(sql)
   # Fetch all the rows in a list of lists.
   results = cursor.fetchall()
   for row in results:
      fname = row[0]
      lname = row[1]
      age = row[2]
      sex = row[3]
      income = row[4]
      # Now print fetched result
      print "fname=%s,lname=%s,age=%d,sex=%s,income=%d" % \
            (fname, lname, age, sex, income )
except:
   print "Error: unable to fecth data"

# disconnect from server
db.close()
```

"SQL": Pandas (Python Library)

- Similar to SQL (declarative), with additional elements of functional programming (map(), filter(), etc.)
- SQL "table" ⇔ Pandas "DataFrame"

Pandas vs. SQL

+ Pandas is lightweight and fast

+ Natively Python, i.e., full SQL expressiveness plus the expressiveness of Python, especially for function evaluation

+ Integration with plotting functions like Matplotlib

- In Pandas, tables must fit into memory

- No post-load indexing functionality: indices are built when a table is created

- No transactions, journaling, etc. (matters for parallel applications)

- Large, complex joins are slower

Document Model

"The world is a hierarchy of entities"

- XML format
- JSON format

Think for a minute:

If we want to use relational DB (e.g., MySQL) instead of XML, how can we store several phone numbers for the same person?

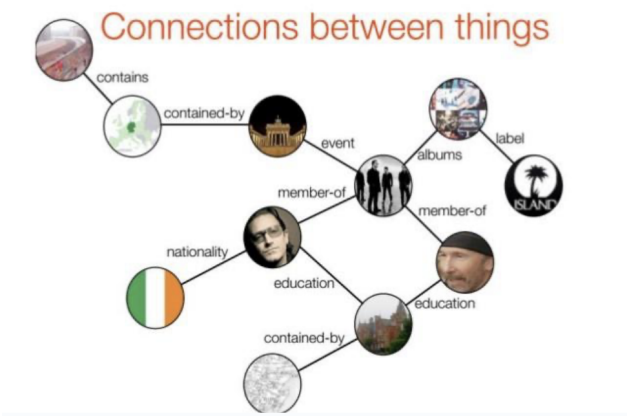
Answer: Separate it in two tables. For instance, a new table of phone numbers. Here, the **Document Model** is more suitable than the **Relational Model** because everything is done in the same place, you just need to change one entry.

Processing XML and JSON

- Document structure = tree
- Processing via tree traversal (depth- or breadth-first search)
- Or use proper query language, such as XQuery or jg

Network Model

"The world is a complex network of entities"



"How should I store my data on a computer?"

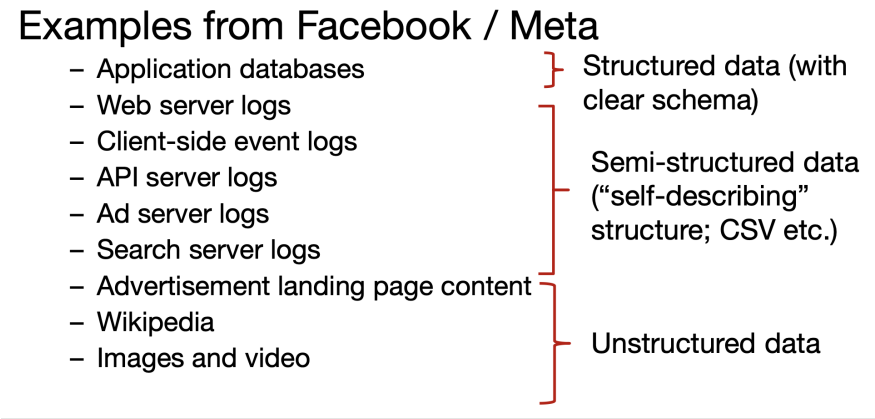
– A word (or two) on binary format

- "Parsing" = converting strings (as stored in text files) to data types used by computer programs (e.g., int, float, boolean, array, list)
- Possibly expensive, but can be avoided by using binary formats: store data to disk "as is", without first converting to strings
- Modern binary formats support nested structures, various levels of schema enforcement, compression, etc.
- Python pickle, Java Serializable, Protocol Buffers (Google), ...

⇒ Consider converting to a binary format at the beginning of your processing pipeline (especially when using "big data"

Data Sources

Data sources at Web companies



Wikipedia

- 300+ languages
- 42 million entities
- Mind-boggling richness of data

How to work with Wikipedia?

- REST API
- XML dumps with wiki markup, SQL database dumps
- Issues: Unicode, size, recency, etc.

Wikidata

- "Database version" of Wikipedia
- {fr: Suisse, de:Schweiz, it:Svizzera, en:Switzerland, ...} $\Rightarrow Q39$
- Both API access and full database dumps
- Available as
  - JSON (document model)
  - RDF (network model)

Web service

- Most large web sites today actively discourage "screen-scraping" to get their content
- Instead: Web service APIs, for interoperable machine-to-machine interaction over a network

- The preferred way to get data from online sources
- Most common framework: REST
  - You request a URL from the server via HTTP
  - The server responds with a text file (e.g., JSON, XML, plaintext)

Data wrangling

Working with raw data sucks

- Data comes in all shapes and sizes
  - CSV files, PDFs, SQL dumps, .jpg, ...

    Different files have different formatting
  - Empty string or space instead of NULL, extra header rows, character encoding...

    "Dirty" data
  - Unwanted anomalies, duplicates

What is data wrangling?

- **Goal:** extract and standardize the raw data
  - Combine multiple data sources
  - Clean data anomalies
- **Strategy:** Combine automation with interactive visualizations to aid in cleaning

Wrangling takes **between 50% and 80% of your time...**

Types of data problems

- Missing data
- Incorrect data
- Inconsistent representation of the same data
- About 75% of data problems require human intervention (e.g., experts, crowdsourcing, etc.)
- Tradeoff between cleaning data vs, over-sanitizing data

Diagnosing data problems

- Visualization and basic stats can convey issues in "raw"data
- Different representations highlight different types of issues:
  - Outliers often stand out in the right kind of plot

  - Missing data will cause gaps or zero values in the right kind of plot
- Becomes increasingly difficult as data gets larger
  - Sampling to the rescue!

Before you start analysing your data

- "Do I have **missing data**?" "If data were missing, how could I know?"
- "Do I have **corrupted data?"** (May arise from measurement errors, processing bugs, etc.)
- **Parse/transform data** into appropriate format for your specific analysis
- Don't be surprised if you need to come back to this stage!

Desiderata

It's always ideal if you can put your hands on the **code/documentation about the dataset** you are analysing (provenance)

It's always ideal if the provided **data format is nicely parseable** (otherwise you need regexes, or maybe even pay humans)

Highly non-parseable data

Entire NY Times archive (since 1851) digitized as of 2015.

$\Rightarrow$ CAPTCHA to prevent spamming and also to parse data from the archives from humans.